

AI 协作反思日志 - Phase 3 详细设计

团队： 暴风星云裂

日期： 2026年5月17日

阶段： P3 详细设计

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
核心类图生成	ChatGPT/Codex	“基于 P1 需求和 P2 架构，为 CampusHub 生成覆盖用户、需求、接单、订单、消息、评价、举报的核心类图，要求标注属性、方法和关系。”	4	35%
SOLID 检查	ChatGPT/Codex	“请按 SOLID 原则审查类图，指出 AI 原始设计中可能违反的地方，并给出修正方案。”	4	30%
API 规范生成	ChatGPT/Codex	“根据需求文档生成 RESTful API 规范，覆盖注册登录、发布需求、浏览需求、接单、订单详情、评价和举报，要求包含请求、响应、错误码。”	4	35%
OpenAPI/YAML 整理	ChatGPT/Codex	“把 Markdown API 文档整理为 OpenAPI 3.0 YAML，保留统一响应、认证方式、分页和枚举定义。”	3	40%
ER 图与建表 SQL	ChatGPT/Codex	“根据核心类图和 API 需要设计 ER 图和 MySQL 建表 SQL，补充索引、外键、隐私字段和并发控制字段。”	4	35%
详细设计整合	Codex	“阅读 P1/P2/P3 全部文档，整合类图、SOLID、API、数据库设计，完成 P3 第 5 个交付物。”	4	25%
反思日志整理	Codex	“按 P3 模板总结本阶段 AI 使用、误导、Prompt 改进和核心工程判断。”	4	30%

2. AI 助力分析（Q1）

本阶段 AI 最大的帮助是把分散的需求、架构、接口和数据库内容快速整理成结构化设计草案。类图、API 表格、错误码、ER 关系和 SQL 初稿都适合由 AI 先生成框架，再由团队根据 P1/P2 约束做人工修订。

具体收益：

- **类图设计**: AI 能快速列出 `User`、`Task`、`Application`、`Order`、`Review`、`Report` 等核心类, 并补充关系方向, 减少从空白开始画图的时间。
- **API 文档**: AI 对 RESTful 路径、请求体、响应体和错误码的标准化表达比较稳定, 适合生成第一版接口规范。
- **数据库设计**: AI 能根据类图自动推导多数外键关系和索引候选, 后续人工再根据查询路径删减或补充。
- **SOLID 检查**: AI 能帮助列出检查维度, 但最终是否真的构成违反, 需要团队结合当前设计阶段和实现方式判断。
- **整合文档**: AI 能把 P1 需求、P2 架构和 P3 交付物串成完整说明, 降低文档整合成本。

估计节省时间: 约 4 小时。节省最多的是 API 初稿和数据库 SQL 初稿, 但这些内容仍需要人工核查一致性。

3. AI 误导分析 (Q2)

SOLID 检查中发现的 AI 设计问题数量: **3 类潜在设计风险**。

最严重的设计问题是: **AI 容易把订单相关能力集中到一个过大的 `Order / OrderService` 中, 使订单状态、聊天、通知、评价、举报联动全部耦合在一起。**

3.1 问题 1: 订单服务容易变成“上帝对象”

AI 初稿倾向:

把确认接单、完成订单、取消订单、发送通知、记录聊天、生成评价入口、处理争议都放在订单对象或订单服务中。

如何发现:

团队对照 P2 的模块依赖图和 P1 的用例走查后发现, 聊天、通知、评价和举报虽然都与订单有关, 但它们不应全部成为订单服务的职责。

人工修正:

- `OrderService` 只负责订单状态流转、订单查询、完成确认、取消和争议入口。
- `ChatService` 负责订单内消息。
- `NotificationService` 负责系统通知。
- `ReviewService / CreditService` 负责评价和信用流水。
- `ReportService` 负责举报目标校验和处理记录。

3.2 问题 2: 需求分类逻辑容易写成大分支

AI 初稿倾向:

虽然能识别 8 类需求, 但容易把所有类别字段校验写到一个 `TaskService` 的发布方法里。

如何发现:

P1 需求中每类任务字段差异较大, 如果所有规则都集中到一个方法, 新增类别或修改规则时会频繁改旧代码, 违反开闭原则。

人工修正:

- 数据库层保留统一 `task` 表和 `category_fields` JSON。
- 业务层引入分类校验策略, 例如 `ExpressTaskValidator`、`SecondHandTaskValidator`、`LostFoundTaskValidator`。
- `TaskService` 只负责调度策略、保存任务和维护状态。

3.3 问题 3：举报目标处理容易混在一个方法里

AI 初稿倾向：

用 `targetType + targetId` 建模举报对象后，容易继续让一个 `ReportService.handle()` 方法处理任务、消息、评价、用户等所有目标。

如何发现：

举报目标虽然统一存储，但不同目标的校验、快照、处罚动作不同。如果一个方法处理全部目标，后续会难以维护。

人工修正：

- 保留统一 `report` 表。
- 处理层按举报目标拆分处理器。
- 管理员处理结果、处罚措施和操作审计分开记录。

3.4 未强行判定为错误的点

本阶段没有把以下设计机械地判定为违反 SOLID：

- `Task` 统一建模不同需求类别不等于违反开闭原则，真正风险在业务校验是否变成大分支。
- `Order` 暴露多个生命周期方法不等于违反单一职责，前提是它们都围绕订单状态本身。
- 当前类图没有复杂继承不是问题，反而避免了不必要的继承层次。
- 类图阶段没有展开所有服务接口，不等于违反依赖倒置，依赖倒置主要在后续 `Controller-Service- Mapper` 分层中落实。

4. Prompt 改进计划（Q3）

上阶段改进措施的验证结果：**有效，但仍需要更明确地要求 AI 输出“约束内设计”，而不是只输出“功能完整设计”。**

P2 阶段提出的改进措施在 P3 中的验证：

P2 改进措施	P3 验证结果
提供上下文锚点	有效。附上 P1 需求和 P2 架构后，AI 输出更贴合 CampusHub，而不是泛化平台模板。
明确不可选项	有效。说明不使用 Redis、消息队列、搜索引擎后，API 和数据库设计更轻量。
要求质量属性评审	有效。SOLID 检查能发现订单服务、任务分类、举报处理的真实风险。
要求方案对比和人工修订	有效。AI 生成初稿后，团队能围绕实现边界做删减。

本阶段新的改进措施：

1. 把“当前阶段是否需要实现”写清楚。
例如收藏、公告、后台管理在数据库和 API 中预留，但编码阶段可以按优先级分批实现。
2. 让 AI 区分“实体设计”和“服务职责”。
类图容易只画实体，后续 Prompt 应要求同时给出 Service 边界，避免实体正确但服务失控。

3. 要求 AI 给出查询路径后再设计索引。

数据库索引不能只根据外键自动生成，应从任务大厅、我的订单、通知列表、举报后台等真实查询路径推导。

4. 要求 AI 标注敏感字段可见性。

例如邮箱、联系方式、取件码、聊天图片和举报证据，不能只设计字段，还要说明谁能看。

5. 要求 AI 输出“删减理由”。

对复杂设计，不只问“怎么做”，还要问“为什么此阶段不做”，避免功能膨胀。

5. 本阶段的核心工程判断

决策内容：

CampusHub P3 详细设计采用“统一任务主表 + JSON 分类字段 + 分类校验策略”的方案；订单状态由 `OrderService` 统一流转；系统通知使用通知工厂集中创建；数据库保持 MySQL 单库，不引入 Redis、消息队列和搜索引擎。

为什么 AI 无法做这个决策：

AI 可以生成多种看起来完整的设计，但无法准确判断本团队的时间、人力、课程验收重点、部署条件和真实用户调研权重。比如 AI 常会建议拆表、缓存、异步事件、复杂审核或完整即时通讯，这些方案在商业系统中合理，但对 3 人学生团队的 MVP 会显著增加风险。

团队最终判断依据是：

- P1 调研显示用户最需要咨询问答、跑腿代取、二手交易、失物招领和站内交流。
- P2 架构已经确定分层单体和 MySQL 单库。
- P3 设计要服务 10 周内可实现的主链路，而不是追求技术复杂度。
- 订单状态一致性和隐私保护比“功能看起来多”更重要。

6. 本阶段人工审查结论

- 已保留 AI 提供的类图结构、API 文档框架、OpenAPI YAML 骨架、ER 图和 DDL 初稿。
- 已人工修正订单服务职责过重、任务分类大分支、举报处理混杂等潜在问题。
- 已补充策略模式和工厂模式，明确它们解决的实际复杂度。
- 已根据 P2 架构约束删除或弱化不必要中间件设计。
- 已统一数据库索引和 API 查询路径，保证任务大厅、订单列表、通知列表、举报后台等核心查询可支撑。
- 已完成 P3 第 5 个交付物 `05-detailed-design.md` 和第 6 个交付物 `06-ai-collaboration-reflection-log3.md`。